

6. File System

1. Introduction to files
 2. Directories
 3. File system implementation
-

6.1 FILES

- 6.1.1 File Naming
- 6.1.2 File Structure
- 6.1.3 File Types
- 6.1.4 File Access
- 6.1.5 File Attributes
- 6.1.6 File Operations

6.2 DIRECTORIES

- 6.2.1 Single-Level Directory Systems
- 6.2.2 Hierarchical Directory Systems
- 6.2.3 Path Names
- 6.2.4 Directory Operations

6.3 FILE-SYSTEM IMPLEMENTATION

- 6.3.1 File-System Layout
 - 6.3.2 Implementing Files
 - 6.3.3 Implementing Directories
 - 6.3.4 Shared Files
 - 6.3.5 Log-Structured File Systems
 - 6.3.6 Journaling File Systems
 - 6.3.7 Virtual File Systems
-

6. File System

1. Introduction to Files

- A file is a collection of similar records.
- A file is a container for a collection of information.
- File represents programs and data. Data files may be numeric, alphabetic, binary or alpha numeric. Files may be free form, such as text files. In general, file is sequence of bits, bytes, lines or records.
- A file has a certain defined structure according to its type.
 - 1 Text File
 - 2 Source File
 - 3 Executable File
 - 4 Object File

Files are the basic units of storage in a computer system. They can contain data, instructions, or information. Files are typically organized into a file system, which allows for efficient storage, retrieval, and management of data. Each file has attributes such as name, type, size, and permissions.

Think of a disk as a linear sequence of fixed-size blocks and supporting reading and writing of blocks. Questions that quickly arise:

- How do you find information?
- How do you keep one user from reading another's data?
- How do you know which blocks are free?

6.1.1 File Naming

Files are identified by names, which are typically human-readable strings. The naming conventions can vary between operating systems.

File Naming

Extension	Meaning
file.bak	Backup file
file.c	C source program
file.gif	Compuserve Graphical Interchange Format image
file.hlp	Help file
file.html	World Wide Web HyperText Markup Language document
file.jpg	Still picture encoded with the JPEG standard
file.mp3	Music encoded in MPEG layer 3 audio format
file.mpg	Movie encoded with the MPEG standard
file.o	Object file (compiler output, not yet linked)
file.pdf	Portable Document Format file
file.ps	PostScript file
file.tex	Input for the TEX formatting program
file.txt	General text file
file.zip	Compressed archive

Figure 4-1. Some typical file extensions.

6.1.2. FILE STRUCTURE

The structure of a file can be:

- **Unstructured:** A sequence of bytes.
- **Structured:** Organized into records or blocks, often used in databases.

File Structure

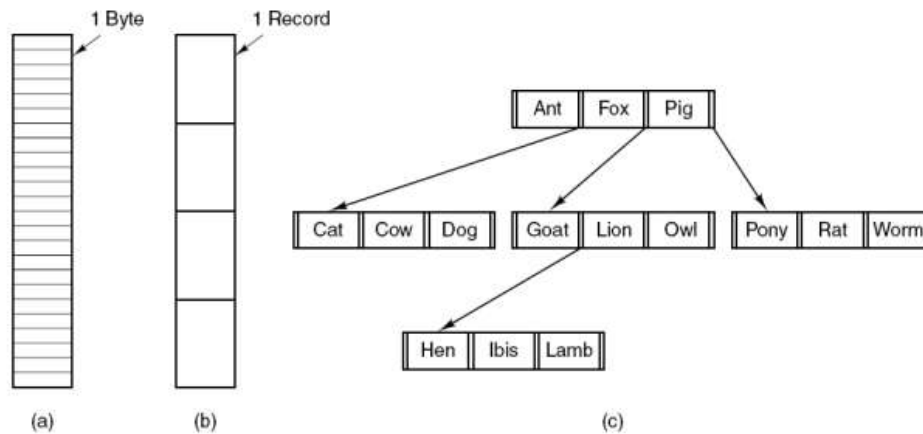


Figure 4-2. Three kinds of files. (a) Byte sequence. (b) Record sequence. (c) Tree.

Files can be structured in several ways, with three common possibilities:

1. **Unstructured Sequence of Bytes:**

- The operating system treats the file as a sequence of bytes without any inherent structure.
- This approach provides maximum flexibility, allowing user programs to impose any structure they need.
- Both UNIX and Windows use this model.

2. **Sequence of Fixed-Length Records:**

- Files are organized as a sequence of fixed-length records, each with some internal structure.
- Historically, this model was used in mainframe systems with 80-character records (similar to punched cards) and 132-character records (for line printers).
- This model is less common in modern general-purpose systems.

3. **Tree of Records:**

- Files consist of a tree of records, each containing a **key** field in a fixed position.
- The tree is sorted on the key field, allowing rapid searching for specific keys.
- This model is used in some large mainframe systems for commercial data processing.

These structures offer different levels of flexibility and efficiency, depending on the needs of the application and the operating system.

6.1.3 File Types

Operating systems support various file types, each serving different purposes:

1. **Regular Files:**

- Contain user information.
- Can be either ASCII (text) files or binary files.
- ASCII files consist of lines of text, which can be displayed and edited easily.
- Binary files have a specific internal structure known to the programs that use them.

2. **Directories:**

- System files that maintain the structure of the file system.
- Help organize files into a hierarchical structure.

3. **Character Special Files:**

- Related to input/output operations.
- Used to model serial I/O devices like terminals, printers, and networks.

4. **Block Special Files:**

- Used to model disks.
- Facilitate block-level operations on storage devices.

Regular Files

- **ASCII Files:** Consist of text lines, easily readable and editable. They can be used in shell pipelines for connecting program outputs to inputs.
- **Binary Files:** Not human-readable, contain specific formats for executable programs or archives. Examples include executable files with headers, text, data, relocation bits, and symbol tables, and archives with library procedures.

File Extensions and Types

- Some systems enforce file types through extensions (e.g., .dat for data files, .c for C programs).
- Strongly typed file systems can cause issues when users perform unexpected operations, leading to restrictions that may frustrate experienced users.

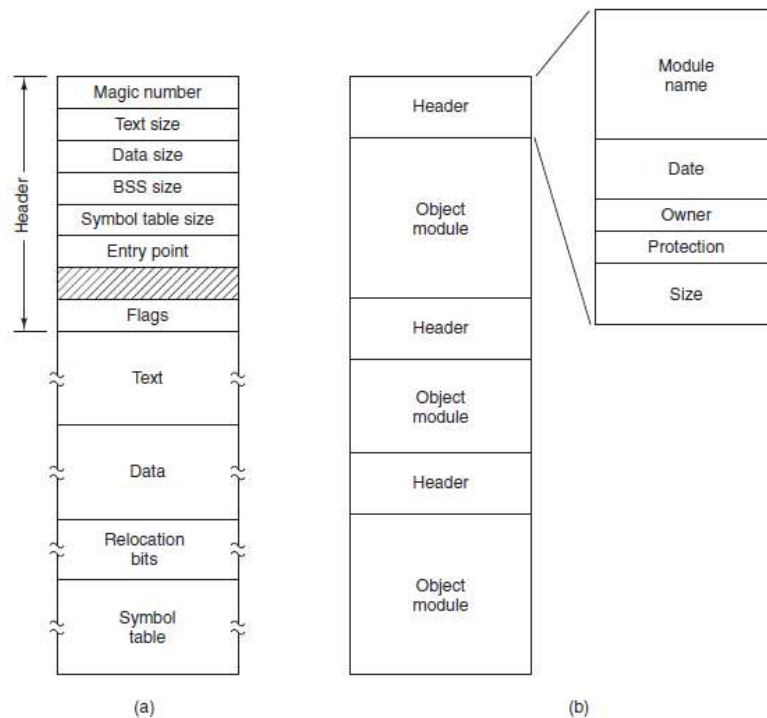


Figure 4-3. (a) An executable file. (b) An archive.

6.1.4 File Access

File access methods have evolved over time, primarily influenced by the storage media used:

1. **Sequential Access:**
 - Early operating systems supported only sequential access.
 - Files could be read from the beginning to the end in order, but not out of order.
 - This method was suitable for magnetic tape storage, where rewinding was possible for repeated reads.
2. **Random Access:**
 - With the advent of disk storage, random access became possible.
 - Files could be read out of order, allowing access to specific bytes or records directly.
 - This method is crucial for applications like database systems, where quick access to specific records is necessary.

Methods for Specifying Read Position

Two primary methods are used to specify where to start reading in a file:

1. **Direct Positioning:**
 - Each read operation specifies the position in the file to start reading from.
2. **Seek Operation:**
 - A special operation called seek sets the current position in the file.
 - Subsequent reads can then proceed sequentially from this position.
 - This method is used in both UNIX and Windows.

6.1.5 FILE ATTRIBUTES

- File attributes vary from one operating system to another. The common attributes are,
 - **Name** – only information kept in human-readable form.
 - **Identifier** – unique tag (number) identifies file within filesystem
 - **Type** – needed for systems that support different types
 - **Location** – pointer to file location on device
 - **Size** – current file size

Files have various attributes, often referred to as metadata, which provide additional information about the file. These attributes can vary significantly between operating systems. Here are some common file attributes:

1. **Protection Attributes:**

- **Access Permissions:** Indicate who can read, write, or execute the file.
- **Password:** Some systems require a password to access certain files.

2. **Flags:**

- **Hidden:** Files that do not appear in standard directory listings.
- **Archive:** Indicates whether the file has been backed up recently.
- **Temporary:** Marks files for automatic deletion when the creating process terminates.

3. **Record Attributes**(for files with records):

- **Record Length:** The length of each record.
- **Key Position:** The position of the key within the record.
- **Key Length:** The length of the key.

4. **Time Stamps:**

- **Creation Time:** When the file was created.
- **Last Access Time:** When the file was last accessed.
- **Last Modification Time:** When the file was last modified.

5. **Size:**

- **Current Size:** The current size of the file.
- **Maximum Size:** Some older systems required specifying the maximum size at creation.

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file was last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

Figure 4-4. Some possible file attributes.

- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk

6.1.6 FILE OPERATIONS

Any file system provides not only a means to store data organized as files, but a collection of functions that can be performed on files. Typical operations include the following:

Files are used to store and retrieve information, and different systems provide various operations to manage files. Here are the most common system calls related to file operations:

1. **Create:**
 - Initializes a new file with no data.
 - Sets initial attributes for the file.
2. **Delete:**
 - Removes a file that is no longer needed.
 - Frees up disk space.
3. **Open:**
 - Prepares a file for access by fetching its attributes and disk addresses into memory.
 - Allows rapid access for subsequent operations.
4. **Close:**
 - Ends access to a file, freeing up internal table space.
 - Ensures the last block of the file is written to disk.
5. **Read:**
 - Retrieves data from a file starting from the current position.
 - Requires specifying the amount of data to read and providing a buffer for the data.
6. **Write:**
 - Adds data to a file at the current position.
 - Increases the file size if writing at the end, or overwrites existing data if writing in the middle.
7. **Append:**
 - Adds data only to the end of the file.
 - A restricted form of write operation.
8. **Seek:**
 - Repositions the file pointer to a specific location within the file.
 - Allows subsequent reads or writes from that position.
9. **Get Attributes:**
 - Retrieves file attributes, such as modification times, which are useful for various applications like the UNIX make program.
10. **Set Attributes:**
 - Allows changing user-settable attributes after the file is created.
 - Includes attributes like protection modes and flags.
11. **Rename:**
 - Changes the name of an existing file.
 - Can often be achieved by copying the file to a new name and deleting the old file.

File Types – Name, Extension

- A common technique for implementing file types is to include the type as part of the file name. The name is split into two parts : a name and an extension. Following table gives the file type with usual extension and function.

File Type	Usual Extension	Function
Executable	exe, com, bin	Read to run machine language program.
Object	obj, o	Compiled, machine language, not linked
Source Code	c, cc, java, pas asm, a	Source code in various language
Text	txt, doc	Textual data, documents

6. 2. Directories

Directories, also known as folders, are used to organize and keep track of files within a file system.

Directories, also known as folders, are special types of files that can contain other files and directories. They help organize files into a hierarchical structure, making it easier to manage and locate files. Directories can have subdirectories, creating a tree-like structure.

They can be structured in various ways to suit different needs.

4.2.1 Single-Level Directory Systems

- **Single Directory:** All files are contained within one directory, often called the root directory.
- **Simplicity:** This system is simple and quick for locating files since there is only one place to look.
- **Usage:** Common in early personal computers and simple embedded devices like digital cameras.

4.2.2 Hierarchical Directory Systems

- **Tree Structure:** Files are organized in a hierarchy of directories, allowing for grouping related files together.
- **User Organization:** Users can create multiple directories and subdirectories to organize their files logically.
- **Example:** A root directory with subdirectories for different users or projects, as shown in Fig. 4-7.

4.2.3 Path Names

- **Absolute Path Names:** Specify the complete path from the root directory to the file. For example:
 - UNIX: /usr/ast/mailbox
 - Windows: \usr\ast\mailbox
 - MULTICS: >usr>ast>mailbox

- **Relative Path Names:** Specify the path relative to the current working directory. For example, if the working directory is /usr/ast, the file mailbox can be referenced simply as mailbox.
- **Working Directory:** Each process has a working directory, which can be changed as needed. Special entries like . (current directory) and .. (parent directory) help navigate the directory tree.

4.2.4 Directory Operations

Directory operations vary across different systems, but here are some common system calls related to directories, using UNIX as an example:

1. **Create:**
 - Creates a new directory.
 - The new directory is empty except for the entries . (current directory) and .. (parent directory).
2. **Delete:**
 - Deletes a directory.
 - Only empty directories (containing only . and ..) can be deleted.
3. **Opendir:**
 - Opens a directory for reading.
 - Necessary for listing all files within the directory.
4. **Closedir:**
 - Closes an open directory.
 - Frees up internal table space.
5. **Readdir:**
 - Returns the next entry in an open directory.
 - Provides a standard format for directory entries, regardless of the internal directory structure.
6. **Rename:**
 - Renames a directory.
 - Similar to renaming files.
7. **Link:**
 - Creates a link to an existing file, allowing it to appear in multiple directories.
 - This is known as a hard link and increments the file's i-node counter.
8. **Unlink:**
 - Removes a directory entry.
 - If the file is only present in one directory, it is removed from the file system. If present in multiple directories, only the specified path is removed.

Symbolic Links

- **Symbolic Links:** Instead of pointing directly to the file's data structure, a symbolic link points to a small file that names another file.
- **Advantages:** Can cross disk boundaries and reference files on remote computers.
- **Disadvantages:** Less efficient than hard links.

6.3 FILE-SYSTEM IMPLEMENTATION

- 6.3.1 File-System Layout
- 6.3.2 Implementing Files
- 6.3.3 Implementing Directories
- 6.3.4 Shared Files
- 6.3.5 Log-Structured File Systems
- 6.3.6 Journaling File Systems
- 6.3.7 Virtual File Systems

FILE-SYSTEM IMPLEMENTATION

1. User vs. Implementor Perspective:

- **Users:** Concerned with file naming, allowed operations, and directory structure.
- **Implementors:** Focus on storage methods, disk space management, and system efficiency and reliability.

2. Key Areas of Focus:

- **Storage of Files and Directories:** How data is organized on the disk.
- **Disk Space Management:** Techniques to allocate and free up space.
- **Efficiency and Reliability:** Ensuring the file system performs well and is robust against failures.

6.3.1 File-System Layout:

- **Partitions:** Disks can be divided into partitions, each with its own file system.
- **MBR (Master Boot Record):** Sector 0 of the disk, used to boot the computer, contains the partition table.
- **Boot Block:** The first block of each partition, used to load the operating system.
- **Superblock:** Contains key parameters about the file system, such as the file-system type and number of blocks.
- **Free Block Information:** Managed using bitmaps or lists of pointers.
- **i-nodes:** Data structures that store information about files.
- **Root Directory:** The top of the file-system tree.
- **Other Directories and Files:** Occupy the remainder of the disk.

6.3.2 Implementing Files

Contiguous Allocation

1. Definition:

- Files are stored in consecutive disk blocks.
- Example: A 50-KB file on a disk with 1-KB blocks would occupy 50 consecutive blocks.

2. Advantages:

- **Simplicity:** Easy to implement by keeping track of the starting block and the number of blocks.

- **Performance:** Excellent read performance as the entire file can be read in a single operation with minimal seek time.

3. Disadvantages:

- **Fragmentation:** Over time, the disk becomes fragmented with free blocks scattered between used blocks.
- **Space Management:** Requires knowing the final size of the file at creation time, which can be impractical for users.

4. Use Cases:

- **CD-ROMs:** Suitable because file sizes are known in advance and do not change.
- **DVDs:** More complex due to file size limitations, often resulting in files being split into multiple contiguous extents.

5. Historical Context:

- Initially used in early magnetic-disk file systems for its simplicity and performance.
- Dropped due to the inconvenience of specifying file size at creation.
- Revived for use in optical media like CD-ROMs and DVDs.

Linked-List Allocation

1. Definition:

- Each file is stored as a linked list of disk blocks.
- The first word of each block is a pointer to the next block, with the rest of the block used for data.

2. Advantages:

- **No External Fragmentation:** Every disk block can be used, avoiding the fragmentation issues seen in contiguous allocation.
- **Simple Directory Entries:** Only the disk address of the first block needs to be stored in the directory entry.

3. Disadvantages:

- **Slow Random Access:** To access block (n), the system must read through the first ($n-1$) blocks sequentially, which is slow.
- **Non-Power-of-Two Block Sizes:** The pointer reduces the amount of data that can be stored in each block, making the block size less efficient for programs that read and write in power-of-two block sizes.
- **Overhead:** Reading full block sizes requires concatenating data from two blocks, adding extra overhead.

Linked-List Allocation Using a Table in Memory (FAT)

1. Definition:

- The pointer word from each disk block is stored in a table in memory, known as the File Allocation Table (FAT).
- Each entry in the FAT corresponds to a disk block and points to the next block in the file.

2. Advantages:

- **Full Block Usage:** The entire disk block is available for data since the pointers are stored in memory.

- **Improved Random Access:** The chain of blocks can be followed in memory without disk references, making random access faster.
- **Simple Directory Entries:** Only the starting block number needs to be stored in the directory entry.
- 3. **Disadvantages:**
 - **Memory Usage:** The entire FAT must be kept in memory, which can be impractical for large disks. For example, a 1-TB disk with 1-KB blocks would require a FAT with 1 billion entries, taking up 2.4 to 3 GB of memory.
 - **Scalability:** The FAT system does not scale well to very large disks due to the significant memory requirements.
- 4. **Historical Context:**
 - The FAT system was originally used in MS-DOS and is still supported by all versions of Windows.

I-nodes

1. **Definition:**
 - An i-node is a data structure that lists the attributes and disk addresses of a file's blocks.
 - Each file has an associated i-node.
2. **Advantages:**
 - **Memory Efficiency:** I-nodes need to be in memory only when the corresponding file is open. This reduces the memory requirement compared to methods like FAT.
 - **Scalability:** The memory required for i-nodes is proportional to the number of open files, not the size of the disk. This makes it scalable for large disks.
3. **Handling Large Files:**
 - If a file grows beyond the fixed number of disk addresses in an i-node, the last address can point to a block containing more disk-block addresses.
 - This can be extended to multiple levels, with blocks pointing to other blocks of addresses.
4. **Usage in File Systems:**
 - **UNIX:** Uses i-nodes extensively, with each file's metadata and block addresses stored in its i-node.
 - **NTFS:** Uses a similar concept with larger i-nodes that can also contain small files.

6.3.3 Implementing Directories

1. **Directory Entries:**
 - **Purpose:** Map the ASCII name of the file to the information needed to locate the data.
 - **Information Stored:** Disk address of the entire file (contiguous allocation), the number of the first block (linked-list schemes), or the i-node number.
2. **Storing Attributes:**
 - **In Directory Entries:** Attributes like owner and creation time are stored directly in the directory entry.
 - **In I-nodes:** Attributes are stored in the i-nodes, making directory entries shorter (just a file name and an i-node number).
3. **File Name Length:**
 - **Fixed-Length Names:** Early systems like MS-DOS and UNIX had short, fixed-length names.
 - **Variable-Length Names:** Modern systems support longer, variable-length names.
4. **Handling Variable-Length Names:**
 - **Fixed Portion with Variable-Length Names:** Directory entries contain a fixed portion followed by the actual file name.

- **Heap Storage:** Directory entries are fixed length, and file names are stored in a heap at the end of the directory.
- 5. **Searching Directories:**
 - **Linear Search:** Directories are searched linearly from beginning to end.
 - **Hash Tables:** Use a hash table to speed up the search process. File names are hashed to select a table entry, and collisions are handled with linked lists.
 - **Caching:** Cache the results of searches to speed up repeated lookups.

4.3.4 Shared Files

1. **File Sharing:**
 - **Links:** A shared file can appear in different directories belonging to different users through links.
 - **DAG Structure:** The file system becomes a Directed Acyclic Graph (DAG) rather than a simple tree.
2. **Challenges:**
 - **Disk Address Copies:** If directories contain disk addresses, appending to the file by one user won't be visible to others.
 - **Ownership and Deletion:** If the file owner deletes the file, linked directories may point to invalid data.
3. **Solutions:**
 - **Data Structure for Disk Blocks:** Store disk blocks in a data structure (like an i-node) associated with the file, and directories point to this structure.
 - **Symbolic Links:** Create a new file of type LINK that contains the path name of the file to which it is linked. This method avoids direct disk address copying.
4. **Drawbacks:**
 - **Hard Links:** Ownership issues and potential invalid pointers if the original file is deleted.
 - **Symbolic Links:** Extra overhead due to additional disk accesses and the need for extra i-nodes and disk blocks to store paths.
5. **Additional Problems:**
 - **Multiple Paths:** Files can have multiple paths, leading to issues like multiple copies during backups and potential duplication when restoring from backups.

4.3.5 Log-Structured File Systems (LFS)

1. **Motivation:**
 - **Technological Changes:** CPUs are getting faster, disks are becoming larger and cheaper, but disk seek times are not improving significantly.
 - **Performance Bottlenecks:** Traditional file systems struggle with small, random writes, leading to inefficiencies.
2. **Design Principles:**
 - **Disk as a Log:** The entire disk is treated as a large log where all writes are appended to the end.
 - **Buffered Writes:** Writes are initially buffered in memory and periodically written to the disk as a single contiguous segment.
3. **Structure:**
 - **Segments:** Each segment contains a mix of i-nodes, directory blocks, and data blocks, along with a segment summary.
 - **I-node Map:** An i-node map is maintained to locate i-nodes, which are scattered throughout the log.
4. **Advantages:**

- **High Write Efficiency:** By writing large segments, the full bandwidth of the disk can be utilized.
 - **Improved Performance:** LFS outperforms traditional UNIX file systems, especially for small writes.
5. **Challenges:**
- **Log Cleaning:** A cleaner thread compacts the log by removing old segments and rewriting live data.
 - **Bookkeeping:** Managing the i-node map and updating i-nodes when blocks are rewritten adds complexity.
6. **Performance:**
- **Small Writes:** LFS significantly improves performance for small writes.
 - **Reads and Large Writes:** Performance is comparable to or better than traditional UNIX file systems.

6.3.6 Journaling File Systems

1. **Purpose:**
- **Robustness:** Maintain a log of planned operations to ensure consistency and recovery after a crash.
2. **Basic Idea:**
- **Logging Operations:** Before performing any file system operations, a log entry is written to disk. If a crash occurs, the system can use the log to complete any pending operations upon reboot.
3. **Example Operation (Removing a File):**
- **Steps:**
 1. Remove the file from its directory.
 2. Release the i-node to the pool of free i-nodes.
 3. Return all the disk blocks to the pool of free disk blocks.
 - **Crash Scenarios:** Crashes at different steps can lead to lost resources or corrupted file pointers.
4. **Journaling Process:**
- **Log Entry:** Write a log entry listing the actions to be completed.
 - **Perform Operations:** Execute the operations only after the log entry is written.
 - **Erase Log Entry:** Remove the log entry after successful completion of operations.
 - **Idempotent Operations:** Ensure that logged operations can be repeated without causing harm.
5. **Atomic Transactions:**
- **Begin and End Transaction:** Group actions into atomic transactions to ensure all or none of the operations are completed.
6. **Examples of Journaling File Systems:**
- **NTFS:** Extensive journaling system, rarely corrupted by crashes.
 - **Linux ext3 and ReiserFS:** Both use journaling, with ext3 maintaining compatibility with the previous ext2 system.

6.3.7 Virtual File Systems

Virtual File Systems (VFS)

1. **Purpose:**
 - **Integration:** Integrate multiple file systems into a single structure, making it seamless for users and processes.
2. **Windows vs. UNIX:**
 - **Windows:** Uses drive letters (C:, D:, etc.) to handle different file systems without integrating them.
 - **UNIX:** Integrates multiple file systems into a single hierarchy, making the presence of different file systems invisible to users.
3. **VFS Structure:**
 - **Upper Interface:** Standard POSIX calls (e.g., open, read, write) from user processes.
 - **Lower Interface:** Function calls to the concrete file systems to perform actual data management.
4. **Key Components:**
 - **Superblock:** Describes a file system.
 - **V-node:** Describes a file.
 - **Directory:** Describes a file system directory.
 - **Mount Table:** Keeps track of mounted file systems.
 - **File Descriptors:** Track open files in user processes.
5. **Operation Example:**
 - **Mounting:** File systems register with the VFS, providing function addresses for required operations.
 - **File Access:** When a file is opened, the VFS locates the appropriate file system, creates a v-node, and sets up the file descriptor.
 - **Read/Write Operations:** The VFS uses the function pointers to call the appropriate functions in the concrete file system.
6. **Flexibility:**
 - **Support for Remote File Systems:** Originally motivated by the need to support NFS (Network File System).
 - **Object-Oriented Design:** Even if implemented in C, the VFS uses object-oriented principles.
7. **Adding New File Systems:**
 - **Function Calls:** New file systems must provide the function calls the VFS requires.
 - **Wrapper Functions:** Existing file systems can be adapted using wrapper functions to meet VFS requirements.

File system implementation involves several components and processes:

- **File Allocation:** Methods like contiguous, linked, and indexed allocation are used to manage how files are stored on disk.
- **Directory Structure:** The organization of directories and files, which can be single-level, two-level, or hierarchical.
- **File Access Methods:** Sequential and direct access methods determine how data within a file is read or written.
- **File Protection:** Mechanisms to control access to files, ensuring security and integrity.
- **File System Mounting:** The process of making a file system accessible at a certain point in the directory tree.
- **File System Maintenance:** Includes tasks like defragmentation, error checking, and backups to ensure the file system remains efficient and reliable.

<https://inf107.telecom-paris.fr/lectures/lecture-part3-4-fs.pdf>

<https://www.cs.fsu.edu/~zwang/files/cop4610/Fall2016/chapter12.pdf>

<https://www.slideshare.net/WajeehaBaig/file-system-237607939>

https://github.com/chlebig/ModernOperatingSystems_AndrewTanenbaum

https://mrcet.com/downloads/digital_notes/IT/R17A0513%20OS.pdf